pfi

Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

SQL Joins and Time Series

WEEK 3 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note: every total ties to the blotter

The Desk Lead's Note

September's blotter is 374 trades and 117,473,749.35 dollars of principal. Tell me how much traded in each grade. Then tell me which benchmark bonds we do not hold, and how much benchmark weight they carry. Last, give me the 10-year's daily changes and month-end levels straight from the database, and show me they match our own functions. Every total has to tie back to the blotter.

The issuers, holdings, trades and spreads are invented; every number describes the data.

```
Work folder pfi_course3_12_k4w7q2xr: 10 data files, Course 2's bondmath.py, marketdata.py as notebook 11 left it, 11 commits, tagged week1 and week2
.............................................   [100%]
46 passed in 0.50s
```

Every join is counted before and after: a total that does not tie is a join that went wrong.

12.1 The Treasury file in long form

Q. The Treasury file has a column per tenor; a time series query wants one "yield_pct" column. Make it long.

```
curve = pd.read_csv('treasury_par_yields.csv', parse_dates=['date'], index_col='date',
                    float_precision='round_trip')
long = curve.reset_index().melt(id_vars='date', var_name='tenor', value_name='yield_pct').dropna()
long['date'] = long['date'].dt.strftime('%Y-%m-%d')
long.to_csv('treasury_long.csv', index=False)
print(f'{len(curve):,} dates x {curve.shape[1]} tenors: {len(long):,} rows with a yield')
2,940 dates x 14 tenors: 35,728 rows with a yield
```

Excel side by side. Power Query does the same melt: select "date", then Transform, Unpivot Columns, Unpivot Other Columns; then Remove Empty from the "yield_pct" filter.

Job	Python	Excel	Result
Every tenor on every date	`.melt(id_vars='date', ...)`	`Table.UnpivotOtherColumns(Wide, {"date"}, "tenor", "yield_pct")`	41,160 rows
Only the rows with a yield	`.dropna()`	`Remove Empty:`	35,728 rows
		`Table.SelectRows(Long, each [yield_pct] <> null and [yield_pct] <> "")`	

Seven tables, and one control figure

Seven Tables Into course3.db

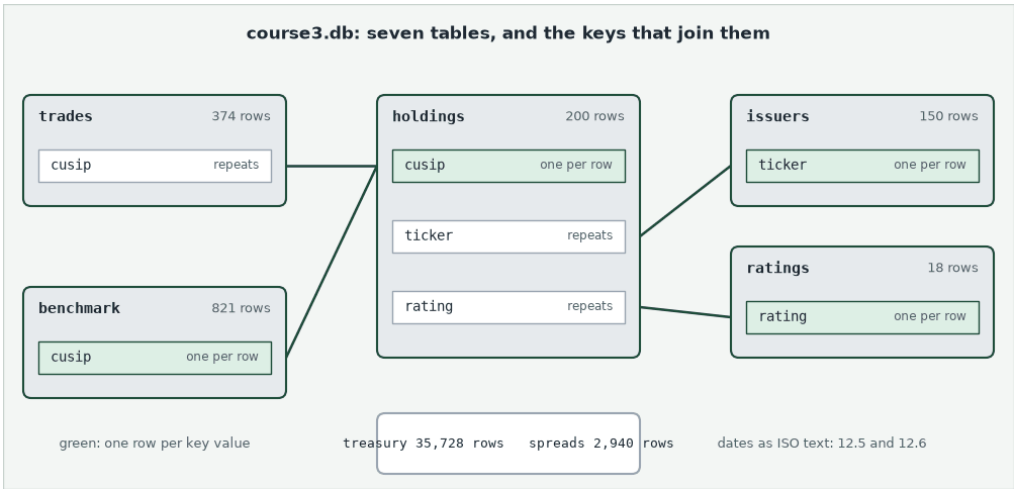
```
db_path = work / 'course3.db'
tables = {'holdings': 'holdings.csv', 'issuers': 'issuers.csv', 'ratings': 'ratings.csv',
          'benchmark': 'benchmark.csv', 'trades': 'trade_blotter.csv',
          'treasury': 'treasury_long.csv', 'spreads': 'course3_synthetic_spreads.csv'}
md.load_tables(db_path, tables)
{'holdings': 200,
 'issuers': 150,
 'ratings': 18,
 'benchmark': 821,
 'trades': 374,
 'treasury': 35728,
 'spreads': 2940}
```

The Control Figure

Q. How many trades, over which dates, and how many dollars?

```
trades = pd.read_csv('trade_blotter.csv')
holdings = pd.read_csv('holdings.csv')
print(f"{len(trades)} trades, {trades['trade_date'].min()} to {trades['trade_date'].max()}")
print(f"principal {trades['principal'].sum():,.2f} dollars")
374 trades, 2026-09-01 to 2026-09-30
principal 117,473,749.35 dollars
```

A control figure is a total every later step must give back. A join that keeps one row per trade keeps this total to the cent.



A trade carries a cusip; its grade sits two tables away. The spreads are the course's synthetic series, not any index.

Seven tables in one file, and one control figure every trade join must keep: 374 trades, 117,473,749.35 dollars.

12.2 JOIN: is the key unique on the one side?

Notebook 11's Exercise 1 wrote one join, "JOIN ratings ON holdings.rating = ratings.rating", as Course 1's merge does in pandas. Today joins are the lesson: short table names, a count before and after, and the rows a join drops. Most desk joins are many to one: many holdings share one issuer, so each finds exactly one issuer row. That is safe only when the key is unique on the "one" side.

Is the Key Unique?

```
md.run_query(db_path, 'SELECT COUNT(*) AS bonds, COUNT(DISTINCT cusip) AS cusips, '
                      'COUNT(DISTINCT ticker) AS tickers FROM holdings')
```

bonds	cusips	tickers
200	200	115

A cusip names one bond: unique in holdings. A ticker names an issuer, and many issuers have several bonds in the book.

Holdings, Issuers and Ratings in One Table

```
book = md.run_query(db_path, '''
    SELECT h.cusip, h.ticker, i.issuer, i.country, h.rating, r.bucket, r.grade, h.market_value
    FROM holdings AS h
    JOIN issuers AS i ON h.ticker = i.ticker
    JOIN ratings AS r ON h.rating = r.rating''')
print(len(book), 'rows')
200 rows
```

"holdings AS h" gives the table a short name, so "h.ticker" is holdings' ticker and "i.ticker" the issuers'. Read "JOIN issuers AS i ON h.ticker = i.ticker" aloud: for each holding, the issuer row with the same ticker. "JOIN" alone is an inner join: a holding with no issuer row would be dropped without a word. 200 in, 200 out: the count is what says none was.

Principal by grade, and the pandas twin

The Same Join in pandas

```
issuers = pd.read_csv('issuers.csv')
ratings = pd.read_csv('ratings.csv')
book_pandas = (holdings[['cusip', 'ticker', 'rating', 'market_value']]
               .merge(issuers[['ticker', 'issuer', 'country']], on='ticker',
                      validate='many_to_one')
               .merge(ratings[['rating', 'bucket', 'grade']], on='rating', validate='many_to_one'))
md.assert_same_rows(book, book_pandas, sort_by=['cusip'])
```

Q. Trades to holdings on cusip, then holdings to ratings: how many trades and dollars in each grade?

```
by_grade = md.run_query(db_path, '''
    SELECT r.grade, COUNT(*) AS trades, SUM(t.principal) AS principal
    FROM trades AS t
    JOIN holdings AS h ON t.cusip = h.cusip
    JOIN ratings AS r ON h.rating = r.rating
    GROUP BY r.grade
    ORDER BY r.grade''')
by_grade.style.hide(axis='index').format({'principal': '{:,.2f}'})
```

grade	trades	principal
High Yield	140	24,176,618.65
Investment Grade	234	93,297,130.70

The pandas Twin, and the Control Figure

```
by_grade_pandas = (trades.merge(book_pandas[['cusip', 'grade']], on='cusip',
                                validate='many_to_one')
                   .groupby('grade')
                   .agg(trades=('trade_id', 'count'), principal=('principal', 'sum'))
                   .reset_index())
md.assert_same_rows(by_grade, by_grade_pandas, sort_by=['grade'])
assert by_grade['trades'].sum() == len(trades)
assert round(by_grade['principal'].sum(), 2) == round(trades['principal'].sum(), 2)
print(f"{by_grade['trades'].sum()} trades, {by_grade['principal'].sum():,.2f} dollars")
```

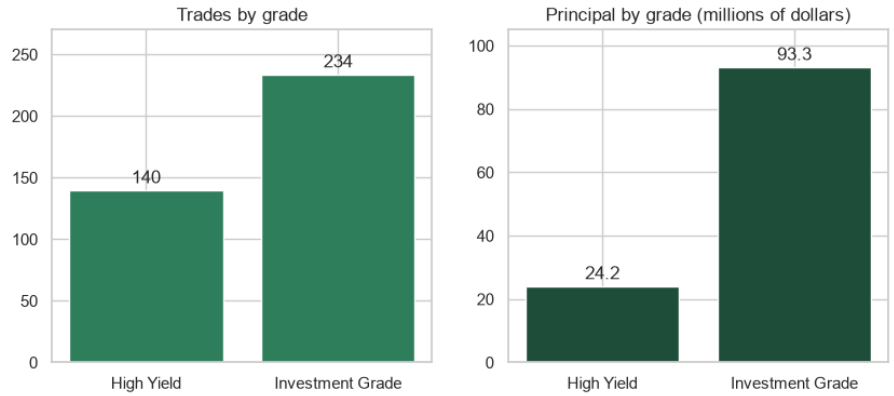
```
374 trades, 117,473,749.35 dollars
```

XLOOKUP and Merge, and trades by grade

Excel side by side. Holdings' cusip, ticker and sector in A2:C201; each trade's cusip in F and ticker in G.

Job	SQL	pandas	Excel	Result
Does a key repeat?	`COUNT(DISTINCT ticker)`	`holdings['ticker'].nunique()`	`=COUNTIF($B$2:$B$201,B2)` filled down	145 rows above 1, largest 6;
				on cusip, largest 1
Each trade's bond, by cusip	`JOIN holdings AS h ON t.cusip = h.cusip`	`merge(on='cusip')`	`=XLOOKUP(F2,$A$2:$A$201,$C$2:$C$201)`	374 trades, no #N/A
			filled down, the sector	
The whole table at once	the same	the same	Power Query:	374 rows
			Home, Merge Queries, Left Outer on cusip	

"XLOOKUP" is a join one column at a time; Power Query's Merge joins whole tables.



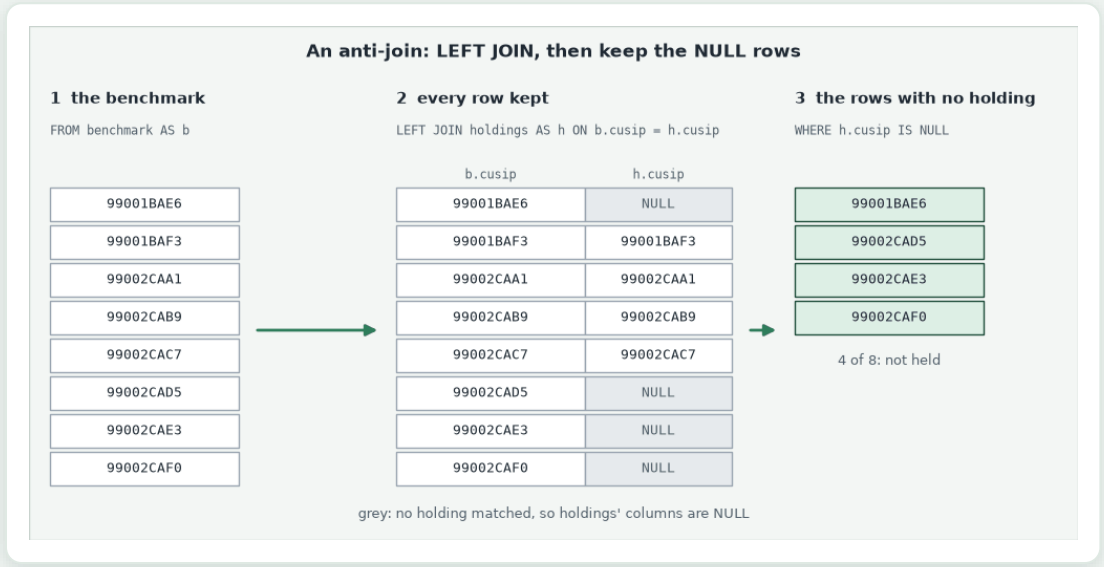
Two joins carried all 374 trades to a grade, and the blotter's 117,473,749.35 dollars tie to the cent.

12.3 LEFT JOIN: the bonds the book does not hold

An inner join drops a row that finds no match. To find the bonds the book does not hold, keep those rows.

```
not_held = md.run_query(db_path, '''
    SELECT b.cusip, b.ticker, b.sector, b.weight_pct
    FROM benchmark AS b
    LEFT JOIN holdings AS h ON b.cusip = h.cusip
    WHERE h.cusip IS NULL''')
print(len(not_held), 'of 821 benchmark bonds are not in the book')
621 of 821 benchmark bonds are not in the book
```

"NULL" is SQL's missing value; pandas reads it as "NaN" or "None". A LEFT JOIN, then the "NULL" rows: this is an anti-join, the rows of one table with no match in the other.



The same rows in pandas, and Left Anti in Excel

The Same Rows in pandas

```
benchmark = pd.read_csv('benchmark.csv')
flagged = benchmark.merge(holdings[['cusip']], on='cusip', how='left', indicator=True,
                          validate='one_to_one')
md.assert_same_rows(not_held, flagged.loc[flagged['_merge'] == 'left_only', not_held.columns],
                    sort_by=['cusip'])
flagged['_merge'].value_counts()
_merge
left_only   621
both   200
right_only   0
Name: count, dtype: int64
```

200 held and 621 not: 821, every benchmark row once.

Excel side by side. Power Query's Merge has this join by name: Home, Merge Queries, then Left Anti, benchmark against holdings on cusip.

Job	SQL	pandas	Excel	Result
Benchmark bonds not held	`LEFT JOIN ... WHERE h.cusip IS NULL`	`indicator=True, 'left_only'`	`JoinKind.LeftAnti`	621 rows

How much weight is not held?

"CASE WHEN ... THEN ... ELSE ... END" gives each row a label, as Excel's "IF" would; "GROUP BY" then adds up each label.

```
coverage = md.run_query(db_path, '''
    SELECT CASE WHEN h.cusip IS NULL THEN 'not held' ELSE 'held' END AS in_book,
           COUNT(*) AS bonds, SUM(b.weight_pct) AS weight_pct
    FROM benchmark AS b
    LEFT JOIN holdings AS h ON b.cusip = h.cusip
    GROUP BY in_book''')
coverage.style.hide(axis='index').format({'weight_pct': '{:.4f}'})
```

in_book	bonds	weight_pct
held	200	24.5577
not held	621	75.4426

```
flagged['in_book'] = 'held'
flagged.loc[flagged['_merge'] == 'left_only', 'in_book'] = 'not held'
coverage_pandas = flagged.groupby('in_book').agg(
    bonds=('cusip', 'count'), weight_pct=('weight_pct', 'sum')).reset_index()
md.assert_same_rows(coverage, coverage_pandas, sort_by=['in_book'])
```

This describes the benchmark's coverage, never what the book should hold.

621 benchmark bonds are not in the book and carry 75.44 percent of the weight: a LEFT JOIN finds what an inner join throws away.

12.4 A key that repeats

Every join so far had a unique key on the "one" side. Now each holding's benchmark weight, joined on the "ticker" both tables share.

Which Key Repeats?

```
repeats = md.run_query(db_path, '''
    SELECT ticker, COUNT(*) AS bonds
    FROM benchmark
    GROUP BY ticker
    HAVING COUNT(*) > 1
    ORDER BY bonds DESC, ticker''')
per_ticker = benchmark.groupby('ticker').agg(bonds=('cusip', 'count')).reset_index()
md.assert_same_rows(repeats, per_ticker[per_ticker['bonds'] > 1], sort_by=['ticker'])
print(len(repeats), 'tickers repeat, up to', repeats['bonds'].max(), 'bonds each')
150 tickers repeat, up to 8 bonds each
```

All 150 benchmark tickers name 3 to 8 bonds: in the benchmark a ticker is never a key; a cusip is.

pandas Refuses

```
holdings.merge(benchmark[['ticker', 'weight_pct']], on='ticker',
validate='many_to_one')
```

```
MergeError: Merge keys are not unique in right dataset; not a
many-to-one merge
```

SQL has no "validate=": its check is the count.

XLOOKUP on a repeated key, and the join on cusip

Excel Side by Side

Job	SQL	pandas	Excel	Result
A trade's bond looked up by ticker		`MergeError with validate=`	`=XLOOKUP(G2,$B$2:$B$201,$A$2:$A$201) filled down`	178 of 374 trades
				get another bond's cusip, no #N/A
The same, on cusip	`ON h.cusip = b.cusip`	`merge(on='cusip')`	`Power Query Merge on cusip`	200 rows

Join on Cusip

```
fixed = md.run_query(db_path, '''
    SELECT h.cusip, h.ticker, h.market_value, b.weight_pct
    FROM holdings AS h
    JOIN benchmark AS b ON h.cusip = b.cusip''')
fixed_pandas = holdings[['cusip', 'ticker', 'market_value']].merge(
    benchmark[['cusip', 'weight_pct']], on='cusip', validate='many_to_one')
md.assert_same_rows(fixed, fixed_pandas, sort_by=['cusip'])
```

```
assert len(fixed) == len(holdings)
assert round(fixed['market_value'].sum(), 2) == round(holdings['market_value'].sum(), 2)
print(len(fixed), 'rows, market value unchanged')
200 rows, market value unchanged
```

12.5 Dates as text: September with BETWEEN

Notebook 11 compared maturities as text: ISO dates, year first, sort in date order.

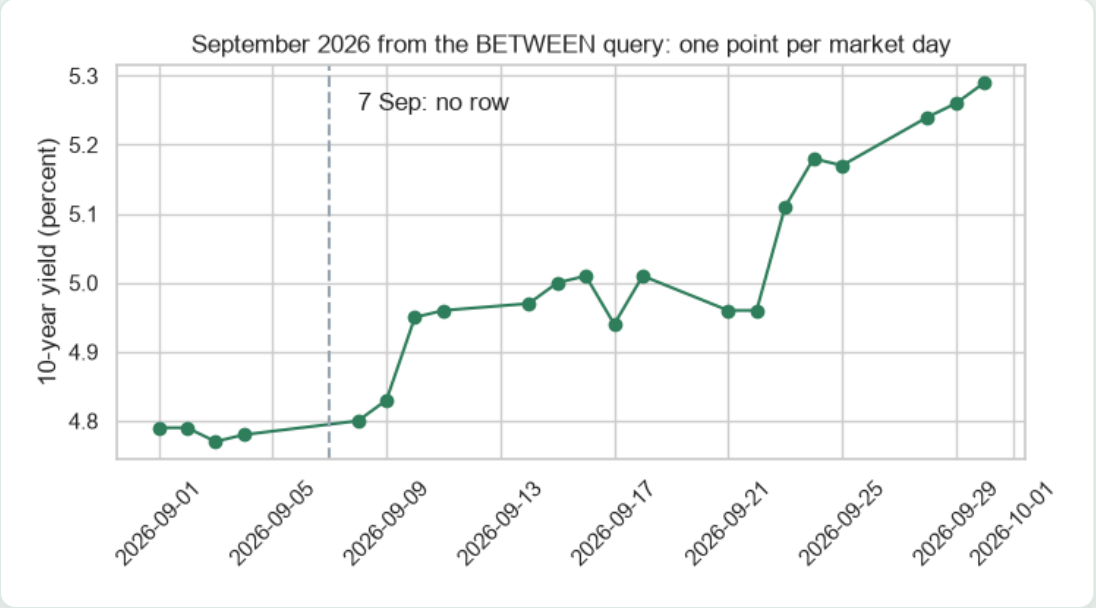
What Is a Date in SQLite?

```
md.run_query(db_path, 'SELECT typeof(date) AS stored_as FROM treasury LIMIT 1')
stored_as
text
```

September With BETWEEN

```
september = md.run_query(db_path, '''
    SELECT date, yield_pct
    FROM treasury
    WHERE tenor = ? AND date BETWEEN ? AND ?
    ORDER BY date''', ('10 Yr', '2026-09-01', '2026-09-30'))
print(len(september), 'market days,', september['date'].iloc[0], 'to', september['date'].iloc[-1])
21 market days, 2026-09-01 to 2026-09-30
```

```
sept = curve.loc['2026-09-01':'2026-09-30', '10 Yr']
september_pandas = pd.DataFrame({'date': sept.index.strftime('%Y-%m-%d'),
                                 'yield_pct': sept.to_numpy()})
md.assert_same_rows(september, september_pandas, sort_by=['date'])
```



These dates carry no time, so '2026-09-30' is the whole last day.

Each month of 2026 with substr

"substr(date, 1, 7)" takes 7 characters from the first: "2026-09", the month.

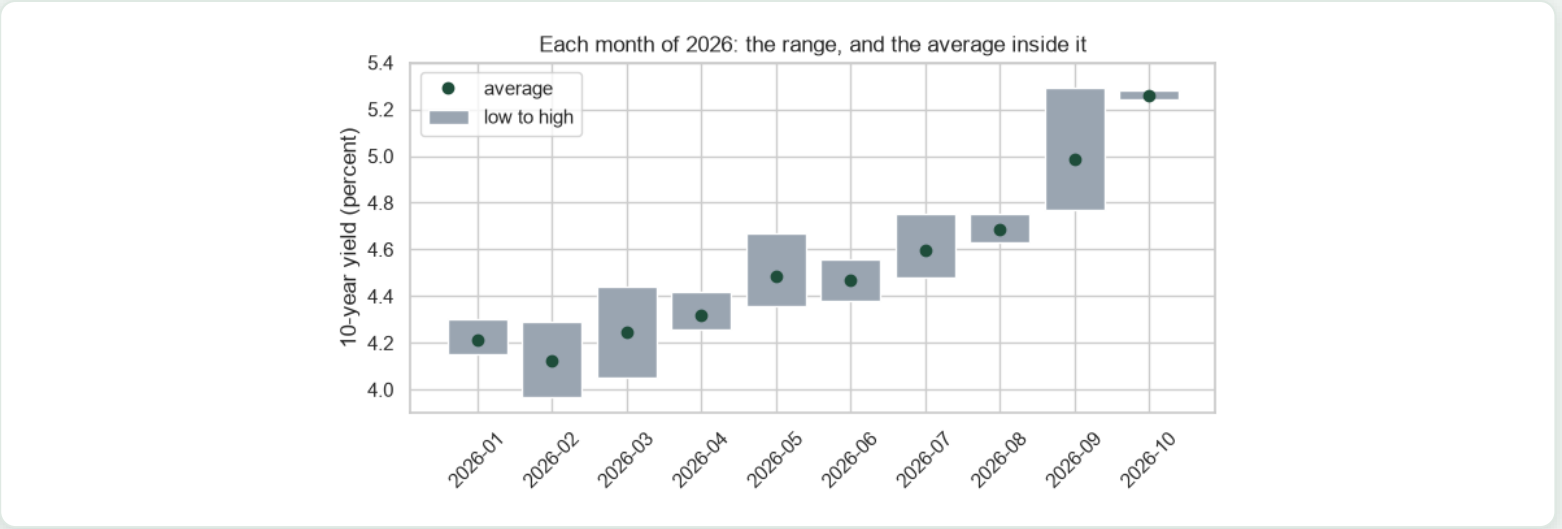
```
months = md.run_query(db_path, '''
    SELECT substr(date, 1, 7) AS month,
           MIN(yield_pct) AS low_pct, MAX(yield_pct) AS high_pct, AVG(yield_pct) AS average_pct
    FROM treasury
    WHERE tenor = ? AND date >= ?
    GROUP BY month
    ORDER BY month''', ('10 Yr', '2026-01-01'))
formats = {'low_pct': '{:.2f}', 'high_pct': '{:.2f}', 'average_pct': '{:.3f}'}
months.style.hide(axis='index').format(formats)
```

```
ten_2026 = curve.loc['2026', '10 Yr']
months_pandas = ten_2026.groupby(ten_2026.index.strftime('%Y-%m')).agg(
    low_pct='min', high_pct='max', average_pct='mean').reset_index()
months_pandas = months_pandas.rename(columns={'date': 'month'})
md.assert_same_rows(months, months_pandas, sort_by=['month'])
```

The averages agree within "assert_same_rows"' tolerance: SQLite and pandas may add in a different order.

month	low_pct	high_pct	average_pct
2026-01	4.15	4.30	4.213
2026-02	3.97	4.29	4.126
2026-03	4.05	4.44	4.246
2026-04	4.26	4.42	4.319
2026-05	4.36	4.67	4.484
2026-06	4.38	4.56	4.470
2026-07	4.48	4.75	4.600
2026-08	4.63	4.75	4.684
2026-09	4.77	5.29	4.989
2026-10	5.24	5.28	5.260

The range of each month, and Excel's date tests



Excel side by side. Dates in A2:A2941, the 10 Yr in B.

Job	SQL	Excel	Result
September's market days	`date BETWEEN ? AND ?`	`=COUNTIFS($A$2:$A$2941,">="&DATE(2026,9,1),$A$2:$A$2941,"<="&DATE(2026,9,30))`	21
September's low	`MIN(yield_pct)`	`=MINIFS($B$2:$B$2941,$A$2:$A$2941,">="&DATE(2026,9,1),$A$2:$A$2941,"<="&DATE(2026,9,30))`	4.77
September's high, average	`MAX, AVG`	`MAXIFS, AVERAGEIFS with the same two date tests`	5.29, 4.989048
The month	`substr(date, 1, 7)`	`=TEXT(DATE(2026,9,30),"yyyy-mm")`	2026-09

12.6 Window functions: LAG

"GROUP BY" collapses many rows into one. A window function keeps every row and lets it look at its neighbours.



"OVER (...)" says what each row may see: "ORDER BY date" puts the rows in date order, and "PARTITION BY" splits them into groups the window never crosses. "DESC" sorts newest first. Window functions need SQLite 3.25 or later; notebook 11 printed 3.50.4.

Excel side by side. "LAG" is the formula you would type anyway: "=(B3-B2)*100", filled down.

```
changes = md.run_query(db_path, '''
    SELECT date, yield_pct,
           (yield_pct - LAG(yield_pct) OVER (ORDER BY date)) * 100 AS change_bp
    FROM treasury
    WHERE tenor = ?
    ORDER BY date''', ('10 Yr',))
changes.tail(3)
```

date	yield_pct	change_bp
2026-09-30	5.29	3.0
2026-10-01	5.24	-5.0
2026-10-02	5.28	4.0

```
change = md.change_bp(curve['10 Yr'])
changes_pandas = pd.DataFrame({'date': change.index.strftime('%Y-%m-%d'),
                               'yield_pct': curve['10 Yr'].to_numpy(),
                               'change_bp': change.to_numpy()})
md.assert_same_rows(changes, changes_pandas, sort_by=['date'])
print(f'LAG equals change_bp on all {len(changes):,} rows')
LAG equals change_bp on all 2,940 rows
```

Each change is from the previous row, whatever the calendar gap: notebook 06's rule. The first row has nothing before it: "NULL" in SQL, "NaN" in pandas.

ROW_NUMBER: the last day of each month

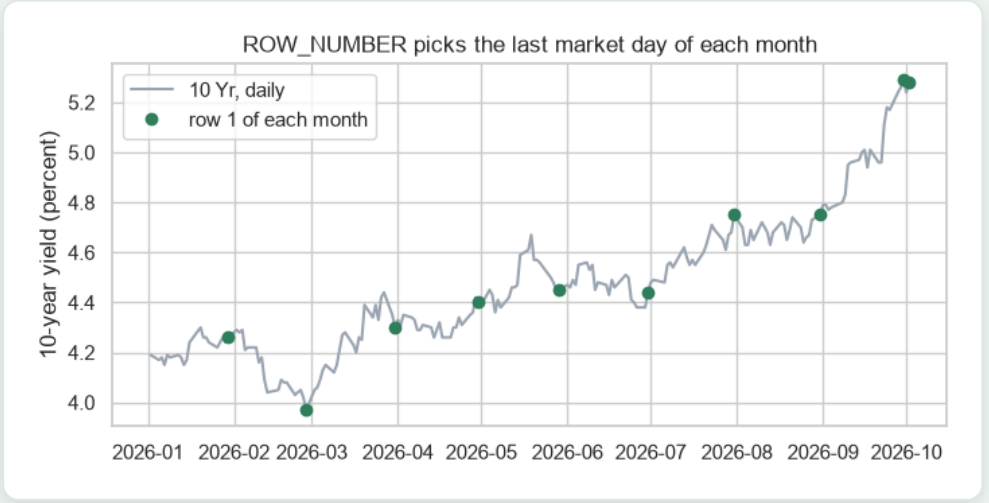
A subquery is a query in brackets, read as a table: the inner query numbers each month's rows from its last day, the outer "WHERE" keeps row 1.

```
month_ends = md.run_query(db_path, '''
    SELECT date, yield_pct FROM (
        SELECT date, yield_pct,
            ROW_NUMBER() OVER (PARTITION BY substr(date, 1, 7) ORDER BY date DESC) AS row_in_month
        FROM treasury
        WHERE tenor = ?
    ) WHERE row_in_month = 1
    ORDER BY date''', ('10 Yr',))
month_ends.tail(3)
```

	date	yield_pct
139	2026-08-31	4.75
140	2026-09-30	5.29
141	2026-10-02	5.28

```
month_end = md.last_per_period(curve['10 Yr'], 'ME')
month_end_pandas = pd.DataFrame({'date': month_end.index.strftime('%Y-%m-%d'),
                                 'yield_pct': month_end.to_numpy()})
md.assert_same_rows(month_ends, month_end_pandas, sort_by=['date'])
print(f'ROW_NUMBER equals last_per_period on all {len(month_ends)} months')
```

```
ROW_NUMBER equals last_per_period on all 142 months
```



October is not over: its row so far is the file's last, 2 October.

Excel side by side. A day of the month in D2:

Job	SQL	Excel	Result
The month's last market date, E2	`ROW_NUMBER() ... ORDER BY date DESC, row 1`	`=MAXIFS($A$2:$A$2941,$A$2:$A$2941,"<="&EOMONTH(D2,0))`	the last row of each month
Its yield	`yield_pct`	`=XLOOKUP(E2,$A$2:$A$2941,$B$2:$B$2941)`	all 12 months of 2025 equal

12.7 Two tests

"Equal on every row" today; make it hold every day. The first test runs the trades join in SQL and pandas and checks the control figure. The second builds its own long Treasury table in pytest's "tmp_path" folder and checks the SQL month ends against "last_per_period".

```
%%writefile -a test_marketdata.py
def test_trades_join_holdings_on_cusip_keeps_rows(tmp_path):
    db_path = tmp_path / "course3.db"
    md.load_tables(db_path, {"trades": HERE / "trade_blotter.csv", "holdings": HERE / "holdings.csv"})
    joined = md.run_query(db_path, "SELECT t.trade_id, t.cusip, t.principal, h.sector FROM trades AS t "
                                   "JOIN holdings AS h ON t.cusip = h.cusip")
    assert len(joined) == 374
    assert round(joined["principal"].sum(), 2) == 117_473_749.35
    trades, holdings = pd.read_csv(HERE / "trade_blotter.csv"), pd.read_csv(HERE / "holdings.csv")
    merged = trades.merge(holdings[["cusip", "sector"]], on="cusip", validate="many_to_one")
    md.assert_same_rows(joined, merged[["trade_id", "cusip", "principal", "sector"]], sort_by=["trade_id"])
```

```
def test_month_end_sql_matches_last_per_period(tmp_path):
    curve = read_treasury()
    long = curve.reset_index().melt(id_vars="date", var_name="tenor", value_name="yield_pct").dropna()
    long["date"] = long["date"].dt.strftime("%Y-%m-%d")
    long.to_csv(tmp_path / "treasury_long.csv", index=False)
    db_path = tmp_path / "course3.db"
    md.load_tables(db_path, {"treasury": tmp_path / "treasury_long.csv"})
    sql = """
        SELECT date, yield_pct FROM (
            SELECT date, yield_pct,
                   ROW_NUMBER() OVER (PARTITION BY substr(date, 1, 7) ORDER BY date DESC) AS row_in_month
            FROM treasury WHERE tenor = ?
        ) WHERE row_in_month = 1
```

```
.............................................   [100%]
48 passed in 0.50s
```

queries.sql, rewritten

Notebook 11's "queries.sql" holds one-table queries. Today's file holds the day's joins, then the time series queries; notebook 11's version stays in the history.

```
%%writefile queries.sql
-- queries.sql: the day's queries, run with ? parameters from Python (marketdata.run_query)
-- every holding with its issuer and its rating's bucket and grade (many to one, twice)
SELECT h.cusip, h.ticker, i.issuer, i.country, h.rating, r.bucket, r.grade, h.market_value
FROM holdings AS h
JOIN issuers AS i ON h.ticker = i.ticker
JOIN ratings AS r ON h.rating = r.rating;
-- trades and principal by grade: trades to holdings on cusip, then to ratings
SELECT r.grade, COUNT(*) AS trades, SUM(t.principal) AS principal
FROM trades AS t
JOIN holdings AS h ON t.cusip = h.cusip
JOIN ratings AS r ON h.rating = r.rating
GROUP BY r.grade
ORDER BY r.grade;
-- benchmark bonds the book does not hold
SELECT b.cusip, b.ticker, b.sector, b.weight_pct
FROM benchmark AS b
LEFT JOIN holdings AS h ON b.cusip = h.cusip
WHERE h.cusip IS NULL;
-- keys that repeat: a ticker with more than one bond in the benchmark
SELECT ticker, COUNT(*) AS bonds
FROM benchmark
GROUP BY ticker
HAVING COUNT(*) > 1
ORDER BY bonds DESC, ticker;
```

```
-- one tenor between two dates; dates are ISO text, and BETWEEN includes both ends
SELECT date, yield_pct
FROM treasury
WHERE tenor = ? AND date BETWEEN ? AND ?
ORDER BY date;
-- the daily change in bp, from the previous row of the same tenor
SELECT date, yield_pct,
       (yield_pct - LAG(yield_pct) OVER (ORDER BY date)) * 100 AS change_bp
FROM treasury
WHERE tenor = ?
ORDER BY date;
-- the last observation of each month, labelled by its own date
SELECT date, yield_pct FROM (
    SELECT date, yield_pct,
           ROW_NUMBER() OVER (PARTITION BY substr(date, 1, 7) ORDER BY date DESC) AS row_in_month
    FROM treasury
    WHERE tenor = ?
) WHERE row_in_month = 1
ORDER BY date;
```

Read the diff, commit the day

```
!git -C "{work}" status --short
!git -C "{work}" diff --stat
 M queries.sql
 M test_marketdata.py
?? treasury_long.csv
 queries.sql    | 61 +++++++
```

Bring it home, and what you can now do

Copy the work folder's "marketdata.py", "test_marketdata.py" and "queries.sql" into your own "my_bondmath" folder, with the trade blotter the new tests read, test them there, and commit. The copy replaces your files, so anything of your own must be in the work folder first. In a PowerShell terminal in VS Code, with the course's ".venv" active:

```
$work = "$env:TEMP\pfi_course3_12_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\marketdata.py", "$work\test_marketdata.py", "$work\queries.sql" .
Copy-Item "$work\trade_blotter.csv" .
python -m pytest -q test_marketdata.py
git status --short
git add marketdata.py test_marketdata.py queries.sql
git commit -m "Notebook 12: tests of joins and month ends, and the day's queries.sql"
```

pytest shows 48 passed. "git status" lists the course's data files as new, "trade_blotter.csv" among them: course data is never committed, so stage the three files by name. In Colab, download the three files from the work folder under "/tmp" instead. The work folder stays in your temp folder; the system clears it in time.

You Can Now

Check a key with "COUNT(DISTINCT ...)" before a join, and join tables with "JOIN ... ON" and table names such as "h".

Find rows with no match with "LEFT JOIN ... WHERE ... IS NULL", and with "indicator=True" in pandas.

Count before and after every join, tie a control figure, and stop a repeated key with "validate=".

Select dates stored as ISO text with "BETWEEN", and group them by month with "substr".

Ask Git which commits changed one file.

Predict first: two joins

Predict 1

A colleague writes "WHERE h.cusip = NULL" in place of "IS NULL". What comes back?

A) the same 621 rows

B) no rows, and no error

C) an error

```
equals_null = md.run_query(db_path, '''
    SELECT b.cusip
    FROM benchmark AS b
    LEFT JOIN holdings AS h ON b.cusip = h.cusip
    WHERE h.cusip = NULL''')
print(len(equals_null), 'rows')
```

Predict 2

200 holdings; the benchmark has 821 bonds from 150 issuers. Join them on ticker. How many rows come back? Write down a number.

```
trap = md.run_query(db_path, '''
    SELECT h.cusip, h.ticker, h.market_value, b.weight_pct
    FROM holdings AS h
    JOIN benchmark AS b ON h.ticker = b.ticker''')
mv_before = holdings['market_value'].sum()
mv_after = trap['market_value'].sum()
print(f'rows:         {len(holdings)} before, {len(trap):,} after')
print(f'market value: {mv_before:,.2f} before, {mv_after:,.2f} after')
print(f"cusips:       {trap['cusip'].nunique()} different")
```

Predict first: dates as text, and a filter

Predict 3

"2026-9-30" is a real day before 2 October, written without its zero. Is '2026-9-30' < '2026-10-02' true?

A) yes, September is before October

B) no

C) an error

```
md.run_query(db_path, "SELECT '2026-09-30' < '2026-10-02' AS iso, "
                      "'2026-9-30' < '2026-10-02' AS no_zero")
```

Predict 4

Only 2026 is wanted, so the date filter goes inside the "LAG" query, beside the tenor. The first change of 2026 is:

A) the same as "change_bp" gives for 2 January

B) empty

C) a change measured from 2015

```
inside = md.run_query(db_path, '''
    SELECT date, (yield_pct - LAG(yield_pct) OVER (ORDER BY date)) * 100 AS change_bp
    FROM treasury
    WHERE tenor = ? AND date >= ?
    ORDER BY date''', ('10 Yr', '2026-01-01'))
inside.head(2)
```

The AI check

The docstring is the prompt. The code that comes back runs, and has one data bug.

1

Docstring

Units, conventions, and what it must never do: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert what the docstring promises. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: "JOIN holdings AS h ON t.ticker = h.ticker" joins on a key that repeats in holdings, so 831 rows for 374 trades.

Code that runs is not the same as code that is right.

The note, answered

The note asked	Answer
Principal by grade	Investment Grade 93,297,130.70 and High Yield 24,176,618.65 dollars, 117,473,749.35 in all
Benchmark bonds not held	621 of 821, 75.44 percent of the weight
Changes and month ends from SQL	Equal to change_bp on 2,940 rows and last_per_period on 142 months

Be proud: you count every join and prove every query against code you trust.

Next: Working With an AI Assistant (notebook 13) puts that habit to work on code you did not write.